EventHub API Documentation

Version: 1.0.0
Author: Nathaniel Sebastian (sc232ns@leeds.ac.uk)
Last Updated: 17th March 2026

Overview

This document describes the REST API for EventHub, the COMP3011 coursework event management system. The API provides endpoints for events, attendees, RSVPs, analytics, and administrative operations.

Base URLs

Environment	URL
Production	`https://comp3011-cw1-api.onrender.com`
Local	`http://127.0.0.1:8000`

Note: On Render's free tier, the first request after 15 minutes of inactivity may take 30–60 seconds to respond while the service spins up.

Authentication

- **Mechanism:** JWT Bearer tokens (HS256)
- **Expiry:** 30 minutes
- **Roles:**
  - **Anonymous:** Read-only access to public endpoints
  - **Authenticated:** CRUD on owned events and owned attendees/RSVPs
  - **Admin:** Access to `/admin/*` endpoints and override rights on protected mutations
- **Rate limiting:** 120 requests/minute (global), 10 requests/minute for `/auth/login`

Include the token in the `Authorization` header:

```
Authorization: Bearer <access_token>
```

Security Headers

All responses include the following headers:

Header	Value
`X-Request-ID`	UUID v4 (request correlation)

Header	Value
`X-Content-Type-Options`	`nosniff`
`X-Frame-Options`	`DENY`
`Referrer-Policy`	`no-referrer`
`Permissions-Policy`	`geolocation=(), microphone=(), camera=()`
`Cross-Origin-Resource-Policy`	`same-site`

ETag Behaviour

- **Endpoints:** `GET /events` and `GET /events/{id}` return an `ETag` header (SHA256 hash of response body)
- **Conditional requests:** Send `If-None-Match: <etag>` to receive `304 Not Modified` with an empty body when content is unchanged
- **Cache headers:** `Cache-Control: no-cache` for ETag-enabled responses; `Cache-Control: no-store` for others

---

Event Ownership

- Events store `created_by_user_id` to track ownership
- **PATCH** and **DELETE** are allowed only for the event owner or an admin
- Non-owners receive `403` with `{"detail": "Not authorised to modify this event"}` or `"Not authorised to delete this event"`
- Events without `created_by_user_id` (imported/legacy) may be modified by any authenticated user

Attendee / RSVP Ownership

- Attendees store `owner_user_id` to track who created them
- `POST /events/{id}/rsvps` is allowed only for the attendee owner or an admin
- `DELETE /events/{id}/rsvps/{rsvp_id}` is allowed for the attendee owner, the owning event creator, or an admin
- Unauthorised RSVP creation returns `403` with `{"detail": "Not authorised to RSVP for this attendee"}`
- Unauthorised RSVP deletion returns `403` with `{"detail": "Not authorised to delete this RSVP"}`

Error Codes

The API may return: `200`, `201`, `204`, `304`, `400`, `401`, `403`, `404`, `409`, `422`, `429`, `500`, `502`

Endpoints

1. Root Redirect

`GET /`

Property	Value
Auth	None
Description	Redirects to `/docs` (OpenAPI documentation)

Response: `302 Found` → `/docs`

---

2. Health Check

`GET /health`

Property	Value
Auth	None
Description	System health check

Response: `200 OK`

```
{
  "status": "online",
  "database": "ok",
  "version": "1.0.0",
  "environment": "prod",
  "commit": "abc123",
  "timestamp": "2026-03-17T12:00:00Z"
}
```

---

3. Register

`POST /auth/register`

Property	Value
Auth	None
Description	Create a new user account

Request body:

```
{
  "username": "johndoe",
  "email": "john@example.com",
  "password": "securepassword"
}
```

Response: `201 Created` (UserOut)

```
{
  "id": 1,
```

```
  "username": "johndoe",
  "email": "john@example.com",
  "created_at": "2026-03-17T12:00:00Z"
}
```

Error codes: `400` (duplicate username/email)

---

4. Login

`POST /auth/login`

Property	Value
Auth	None
Description	Obtain JWT access token
Content-Type	`application/x-www-form-urlencoded`

Request body (form data):

```
username=johndoe&password=securepassword
```

Response: `200 OK`

```
{
  "access_token": "eyJhbGciOiJIUzI1NiIsInR5cCI6IkpXVCJ9...",
  "token_type": "bearer"
}
```

Error codes: `401` (invalid credentials), `429` (rate limit exceeded)

---

5. List Events

`GET /events`

Property	Value
Auth	None
Description	Paginated list of events
ETag	Yes

Query parameters:

Parameter	Type	Default	Description
`q`	string	—	Search query
`location`	string	—	Filter by location

Parameter	Type	Default	Description
`start_after`	datetime	—	Events starting after this time
`start_before`	datetime	—	Events starting before this time
`limit`	int	10	Page size (1–100)
`offset`	int	0	Pagination offset
`sort`	string	—	Sort field
`min_capacity`	int	—	Minimum capacity
`status`	string	—	`upcoming` or `past`

Example request:

```
GET /events?limit=10&offset=0&status=upcoming
```

Response: `200 OK` (PaginatedResponse)

```
{
  "items": [
    {
      "id": 1,
      "title": "Tech Meetup",
      "description": "Monthly tech meetup",
      "location": "Leeds",
      "start_time": "2026-04-01T18:00:00Z",
      "end_time": "2026-04-01T20:00:00Z",
      "capacity": 50,
      "created_at": "2026-03-01T10:00:00Z"
    }
  ],
  "total": 42,
  "limit": 10,
  "offset": 0
}
```

Response headers: `ETag`, `Cache-Control: no-cache`

Error codes: `422` (validation error)

---

6. Create Event

`POST /events`

Property	Value
Auth	Required
Description	Create a new event. Stores `created_by_user_id` for ownership

Request body:

```
{
  "title": "Tech Meetup",
  "description": "Monthly tech meetup",
  "location": "Leeds",
  "start_time": "2026-04-01T18:00:00Z",
  "end_time": "2026-04-01T20:00:00Z",
  "capacity": 50
}
```

Response: `201 Created` (EventOut)

```
{
  "id": 1,
  "title": "Tech Meetup",
  "description": "Monthly tech meetup",
  "location": "Leeds",
  "start_time": "2026-04-01T18:00:00Z",
  "end_time": "2026-04-01T20:00:00Z",
  "capacity": 50,
  "created_at": "2026-03-17T12:00:00Z"
}
```

Error codes: `401` (unauthorized), `422` (validation error)

---

7. Get Event

`GET /events/{id}`

Property	Value
Auth	None
Description	Retrieve a single event by ID
ETag	Yes

Path parameters: `id` (integer)

Example request:

```
GET /events/1
```

Response: `200 OK` (EventOut)

```
{
  "id": 1,
  "title": "Tech Meetup",
  "description": "Monthly tech meetup",
  "location": "Leeds",
  "start_time": "2026-04-01T18:00:00Z",
  "end_time": "2026-04-01T20:00:00Z",
  "capacity": 50,
  "created_at": "2026-03-01T10:00:00Z"
}
```

Response headers: `ETag`, `Cache-Control: no-cache`

Error codes: `404` (event not found)

---

8. Get Event Provenance

`GET /events/{id}/provenance`

Property	Value
Auth	None
Description	Retrieve provenance metadata for an event (source, import run, hash, etc.)

Path parameters: `id` (integer)

Example request:

```
GET /events/1/provenance
```

Response: `200 OK` (EventProvenanceOut)

```
{
  "event_id": 1,
  "source_name": "Leeds Events XML",
  "source_url": "https://example.com/events.xml",
  "source_record_id": "evt-123",
  "import_run_id": 5,
  "imported_at": "2026-03-15T10:00:00Z",
  "latest_import_at": "2026-03-15T10:00:00Z",
  "parser_version": "1.0",
  "sha256_hash": "a1b2c3d4e5f6...",
  "is_seeded": false,
  "is_user_created": false
}
```

Error codes: `404` (event not found)

---

9. Update Event

`PATCH /events/{id}`

Property	Value
Auth	Required (owner or admin)
Description	Partially update an event

Path parameters: `id` (integer)

Request body (partial EventUpdate):

```
{
  "title": "Updated Tech Meetup",
  "capacity": 75
}
```

Response: `200 OK` (EventOut)

Error codes: `401` (unauthorized), `403` (not owner/admin), `404` (event not found), `422` (validation error)

---

10. Delete Event

`DELETE /events/{id}`

Property	Value
Auth	Required (owner or admin)
Description	Delete an event

Path parameters: `id` (integer)

Response: `204 No Content`

Error codes: `401` (unauthorized), `403` (not owner/admin), `404` (event not found)

---

11. Get Event Stats

`GET /events/{id}/stats`

Property	Value
Auth	None
Description	RSVP counts and remaining capacity

Path parameters: `id` (integer)

Example request:

```
GET /events/1/stats
```

Response: `200 OK`

```
{
  "event_id": 1,
  "going": 12,
  "maybe": 3,
  "not_going": 1,
  "remaining_capacity": 34
}
```

Error codes: `404` (event not found)

---

12. List Event RSVPs

`GET /events/{id}/rsvps`

Property	Value
Auth	None
Description	List all RSVPs for an event

Path parameters: `id` (integer)

Response: `200 OK` (List[RSVPOut])

```
[
  {
    "id": 1,
    "event_id": 1,
    "attendee_id": 5,
    "status": "going",
    "created_at": "2026-03-16T14:00:00Z"
  }
]
```

Error codes: `404` (event not found)

---

13. Create RSVP

`POST /events/{id}/rsvps`

Property	Value
Auth	Required
Description	Create an RSVP for an event using an attendee owned by the current user (or admin override)

Path parameters: `id` (integer)

Request body:

```
{
  "attendee_id": 5,
  "status": "going"
}
```

Status values: `going`, `maybe`, `not_going`

Response: `201 Created` (RSVPOut)

```
{
  "id": 1,
  "event_id": 1,
  "attendee_id": 5,
  "status": "going",
  "created_at": "2026-03-17T12:00:00Z"
}
```

Error codes: `401` (unauthorized), `403` (attendee not owned by current user), `404` (event or attendee not found), `409` (duplicate RSVP), `422` (validation error)

---

14. Delete RSVP

`DELETE /events/{id}/rsvps/{rsvp_id}`

Property	Value
Auth	Required
Description	Remove an RSVP. Allowed for attendee owner, owning event creator, or admin

Path parameters: `id` (integer), `rsvp_id` (integer)

Response: `204 No Content`

Error codes: `401` (unauthorized), `403` (not authorised), `404` (not found)

---

15. Create Attendee

`POST /attendees`

Property	Value
Auth	Required
Description	Create a new attendee profile and store `owner_user_id` as the current user

Request body:

```
{
  "name": "Jane Smith",
  "email": "jane@example.com"
}
```

Response: `201 Created` (AttendeeOut)

```
{
  "id": 1,
  "name": "Jane Smith",
  "email": "jane@example.com"
}
```

Error codes: `401` (unauthorized), `409` (duplicate email), `422` (validation error)

---

16. Get Attendee

`GET /attendees/{id}`

Property	Value
Auth	None
Description	Retrieve an attendee by ID

Path parameters: `id` (integer)

Response: `200 OK` (AttendeeOut)

```
{
  "id": 1,
  "name": "Jane Smith",
  "email": "jane@example.com",
  "created_at": "2026-03-17T12:00:00Z"
}
```

Error codes: `404` (attendee not found)

---

17. Get Attendee Events

`GET /attendees/{id}/events`

Property	Value
Auth	None
Description	List events the attendee has RSVP'd to

Path parameters: `id` (integer)

Response: `200 OK` (List[EventOut])

Error codes: `404` (attendee not found)

---

18. Seasonality Analytics

`GET /analytics/events/seasonality`

Property	Value
Auth	None
Description	Monthly event distribution with top locations per month

Response: `200 OK`

```
{
  "items": [
    {
      "month": "2026-04",
      "count": 15,
      "top_locations": ["Leeds", "Manchester", "York"]
    }
  ]
}
```

Uses dialect-aware SQL (SQLite `strftime` / PostgreSQL `to_char`).

---

19. Trending Events

`GET /analytics/events/trending`

Property	Value
Auth	None
Description	Events ranked by trending score based on recent and total RSVPs

Query parameters:

Parameter	Type	Default	Description
`window_days`	int	30	Days to consider for "recent" RSVPs
`limit`	int	5	Number of results

Trending score: `(recent_rsvps * 1.5) + (total_rsvps * 0.5)`

Response: `200 OK`

```
[
  {
    "event_id": 3,
    "title": "Popular Conference",
    "trending_score": 42.5,
    "recent_rsvps": 25
  }
]
```

---

20. Event Recommendations

`GET /events/recommendations`

Property	Value
Auth	Required

Property	Value
Description	Location-based personalised event recommendations

Response: `200 OK`

```
{
  "recommendations": [
    {
      "event_id": 2,
      "title": "Local Workshop",
      "score": 0.85,
      "reason": "Matches your location preferences",
      "location": "Leeds",
      "start_time": "2026-04-15T10:00:00Z"
    }
  ],
  "user_id": 1
}
```

Error codes: `401` (unauthorized)

---

21. Run Import

`POST /admin/imports/run`

Property	Value
Auth	Admin only
Description	Trigger an import from an external source

Query parameters:

Parameter	Type	Required	Description
`source_type`	string	Yes	`xml` or `csv`
`source_url`	string	Yes	URL of the data source

Example request:

```
POST /admin/imports/run?source_type=xml&source_url=https://example.com/events.xml
```

Response: `201 Created`

Error codes: `403` (non-admin), `502` (import failure)

---

22. List Imports

`GET /admin/imports`

Property	Value
Auth	Admin only
Description	List import run history

Response: `200 OK` (List[ImportRunOut])

Error codes: `403` (non-admin)

---

23. Dataset Metadata

`GET /admin/dataset/meta`

Property	Value
Auth	Admin only
Description	Dataset metadata or message if no dataset imported

Response: `200 OK`

Returns dataset metadata object or `"No dataset imported yet"`.

Error codes: `403` (non-admin)

---

24. Import Quality Report

`GET /admin/imports/quality`

Property	Value
Auth	Admin only
Description	Import quality metrics and run details

Response: `200 OK`

```
{
  "total_runs": 10,
  "successful_runs": 8,
  "failed_runs": 2,
  "success_rate": 0.8,
  "last_successful_import": "2026-03-15T10:00:00Z",
  "avg_duration_ms": 300000,
  "total_rows_read": 1500,
  "total_rows_inserted": 1200,
  "total_rows_updated": 300,
  "recent_failures_count": 2,
  "runs": [
    {
      "id": 5,
      "data_source_id": 1,
      "status": "success",
      "started_at": "2026-03-15T09:00:00Z",
      "finished_at": "2026-03-15T09:05:00Z",
```

```
      "rows_read": 150,
      "rows_inserted": 120,
      "rows_updated": 30,
      "duration_ms": 300000,
      "error_count": 0,
      "sha256_hash": "a1b2c3...",
      "parser_version": "1.0"
    }
  ]
}
```

Error codes: `403` (non-admin)

Running Locally

```
git clone https://github.com/NathS04/comp3011-cw1-api.git && cd comp3011-cw1-api
python3 -m venv venv && source venv/bin/activate
pip install -r requirements.txt
export DATABASE_URL="sqlite:///./app.db" SECRET_KEY="dev-secret"
alembic upgrade head
pytest -q   # Expected: 84 passed
uvicorn app.main:app --reload
```

Testing

The test suite comprises 84 tests with verified 95% app coverage. Run tests with:

```
pytest -q
pytest --cov=app --cov-report=term-missing
```

Expected output: `84 passed`

End-to-End Example

This example walks through a typical workflow: register, login, create an event, create an attendee, RSVP, use analytics, ETag caching, and provenance.

1. Register

```
curl -X POST http://127.0.0.1:8000/auth/register \
  -H "Content-Type: application/json" \
  -d '{"username":"alice","email":"alice@example.com","password":"secret123"}'
```

2. Login

```
curl -X POST http://127.0.0.1:8000/auth/login \
  -d "username=alice&password=secret123"
```

Save the `access_token` from the response.

3. Create Event

```
curl -X POST http://127.0.0.1:8000/events \
  -H "Authorization: Bearer <access_token>" \
  -H "Content-Type: application/json" \
  -d '{
    "title":"Workshop",
    "description":"Hands-on session",
    "location":"Leeds",
    "start_time":"2026-04-20T14:00:00Z",
    "end_time":"2026-04-20T16:00:00Z",
    "capacity":30
  }'
```

Note the event `id` from the response (e.g. `1`).

4. Create Attendee

```
curl -X POST http://127.0.0.1:8000/attendees \
  -H "Authorization: Bearer <access_token>" \
  -H "Content-Type: application/json" \
  -d '{"name":"Alice","email":"alice@example.com"}'
```

Note the attendee `id` (e.g. `1`).

5. RSVP

```
curl -X POST http://127.0.0.1:8000/events/1/rsvps \
  -H "Authorization: Bearer <access_token>" \
  -H "Content-Type: application/json" \
  -d '{"attendee_id":1,"status":"going"}'
```

6. Analytics

```
# Seasonality
curl http://127.0.0.1:8000/analytics/events/seasonality

# Trending
curl "http://127.0.0.1:8000/analytics/events/trending?window_days=30&limit=5"

# Recommendations (requires auth)
curl http://127.0.0.1:8000/events/recommendations \
  -H "Authorization: Bearer <access_token>"
```

7. ETag Caching

```
# First request — returns full response and ETag
curl -i http://127.0.0.1:8000/events/1

# Second request with If-None-Match — returns 304 if unchanged
curl -i http://127.0.0.1:8000/events/1 \
  -H "If-None-Match: <etag_from_previous_response>"
```

8. Provenance

```
curl http://127.0.0.1:8000/events/1/provenance
```

Returns source metadata, import run details, SHA256 hash, and whether the event was user-created or imported.